# Practice Questions based on Constructors

08 November 2026 10:07

Constructors initialize objects, must share the class name, have no return type, and overloaded constructors must differ by parameter list.

**Practice questions**

**1. Student record system**

Create a class Student with fields name, rollNumber, marks, and grade.

- Write a no-argument constructor that sets name to "Unknown", rollNumber to 0, marks to 0.0, and grade to "N/A".
- Write a parameterized constructor that accepts all four values.
- Write a constructor that accepts only name and rollNumber; it should set marks to 0.0 and grade to "N/A".
- Use this() constructor chaining where appropriate.
- Add a method calculateGrade() that assigns a grade based on marks: 90+ = "A", 75–89 = "B", 60–74 = "C", 40–59 = "D", otherwise "F".
- In main(), create at least three students using different constructors and print their details.

**2. Bank account with validation**

Create a class BankAccount with accountHolder, accountNumber, balance, and accountType.

- Write a default constructor that creates an account with holder "Unnamed", account number "000000", balance 0.0, and type "Savings".
- Write a parameterized constructor for all fields.
- Write a constructor accepting only accountHolder and accountNumber; assign type "Savings" and balance 0.0.
- Prevent a negative initial balance by throwing an IllegalArgumentException in the constructor.
- Add methods deposit(double amount), withdraw(double amount), and displayAccountDetails().
- Ensure withdrawal fails safely if the amount is negative or exceeds the balance.
- In main(), demonstrate all constructors and both successful and unsuccessful transactions.

**3. Book inventory**

Create a class Book with fields title, author, isbn, price, and quantity.

- Implement four overloaded constructors:
  - No arguments.
  - title and author only.
  - title, author, and price.
  - All fields.
- Use constructor chaining so that every constructor ultimately calls the full parameterized constructor.
- If price is negative, set it to 0.0 and print a warning.
- If quantity is negative, set it to 0.
- Add methods addStock(int quantity), sellCopies(int quantity), and displayBook().

- In main(), create books using every constructor and test stock updates.

**4. Employee payroll system**

Create a class Employee with employeeId, name, department, basicSalary, and hra.

- Write a no-argument constructor using sensible default values.
- Write a constructor taking employeeId, name, and department; set basicSalary to 15000 and hra to 10% of basic salary.
- Write a constructor taking all fields.
- Validate that basicSalary is not negative; if it is, throw an exception.
- Add a method calculateTotalSalary() that returns basicSalary + hra.
- Add a method applyIncrement(double percentage) that increases basic salary and recalculates HRA.
- In main(), create employees through each constructor, print salary details, apply an increment, and print updated details.

**5. Rectangle geometry**

Create a class Rectangle with length, width, and a nested or separate class Point having x and y.

- Create a Point class with a no-argument constructor setting both coordinates to 0, and a parameterized constructor.
- Create a Rectangle class with:
  - A no-argument constructor that sets length and width to 1 and origin to (0, 0).
  - A constructor accepting length and width.
  - A constructor accepting a Point object, length, and width.
- Validate that length and width are positive; otherwise throw an exception.
- Add methods area(), perimeter(), move(int newX, int newY), and displayDetails().
- In main(), create rectangles using all constructors and calculate their area and perimeter.

**6. Copy constructor for a product**

Create a class Product with fields productId, productName, price, and stockQuantity.

- Write a parameterized constructor.
- Write a **copy constructor** that accepts another Product object and copies all its values.
- Modify the copy without changing the original product, then print both objects to prove they are independent.
- Add methods applyDiscount(double percentage) and updateStock(int change).
- Ensure discount percentage is between 0 and 100; otherwise throw an exception.
- In main(), create one product, make a copy, apply a discount only to the copy, and display both products.

**7. Constructor chaining in a vehicle hierarchy**

Create a class Vehicle with fields brand, model, year, and price.

- Write a no-argument constructor that calls a parameterized constructor using this(...).
- Write a constructor accepting brand and model.
- Write a constructor accepting all fields.

- Use this() chaining so that common initialization is written only once.
- Add validation: year must be between 1900 and the current year, and price must be non-negative.
- Add methods displayVehicle() and calculateResaleValue() that returns 60% of price if the vehicle is more than 5 years old, otherwise 80%.
- In main(), create vehicles using every constructor and display their resale values.

**8. Library membership system**

Create a class LibraryMember with fields memberId, memberName, booksIssued, and fineAmount.

- Write a default constructor that sets memberId to "LIB-000", name to "New Member", books issued to 0, and fine to 0.0.
- Write a constructor accepting memberId and memberName.
- Write a constructor accepting all fields.
- A member may issue a maximum of 5 books. Add a method issueBook() that increases booksIssued only if the limit has not been reached.
- Add a method returnBook() that decreases the count only if it is greater than zero.
- Add a method addFine(double amount) that rejects negative amounts.
- In main(), simulate issuing and returning books, then display the member's final status.

**9. Immutable point with constructor overloading**

Create a class Point with private final int x and private final int y.

- Write a no-argument constructor that sets both coordinates to 0.
- Write a constructor accepting x and y.
- Write a copy constructor accepting another Point.
- Make the class immutable: provide only getter methods and no setters.
- Add methods distanceFromOrigin() and distanceFrom(Point other).
- Override toString() to return a readable representation such as Point(x=3, y=4).
- In main(), create points using all constructors and calculate distances between them.

**10. Online course enrollment**

Create a class Course with fields courseId, courseName, instructor, fee, capacity, and enrolledStudents.

- Write a no-argument constructor that creates a course with capacity 30, fee 0.0, and no enrolled students.
- Write a constructor accepting courseId, courseName, and instructor.
- Write a constructor accepting all fields.
- Validate that capacity is greater than zero and fee is non-negative.
- Add methods:
  - enrollStudent(), which succeeds only if seats are available.
  - withdrawStudent(), which succeeds only if at least one student is enrolled.
  - availableSeats().
  - displayCourseDetails().
- In main(), create courses using different constructors, enroll students until the course is full, attempt one extra enrollment, and then withdraw a student.

**Concepts covered**

| Question | Main constructor concept |
|---|---|
| 1 | Default and parameterized constructors |
| 2 | Validation and exception handling |
| 3 | Constructor overloading and chaining |
| 4 | Derived values and validation |
| 5 | Object composition with constructors |
| 6 | Copy constructor |
| 7 | this() constructor chaining |
| 8 | State constraints and object behavior |
| 9 | Immutability and copy construction |
| 10 | Real-world object initialization |

A constructor is invoked by new when an object is created; Java differentiates overloaded constructors by their parameter lists, and this(...) must be the first statement when one constructor calls another.